

ROBOT OPERATING SYSTEM

DATE: 7/10/2025

By Saumya Arya

INTRODUCTION

Robots are becoming a big part of our everyday life, from cleaning our homes to helping in factories and even performing surgeries. But for a robot to work properly, all its parts like sensors, motors, and programs need to communicate with each other smoothly. This is where the Robot Operating System, or ROS, comes in.

ROS is not really an operating system like Windows or Linux. It is more like a smart helper that gives tools and libraries to make building and controlling robots easier. It helps different parts of a robot talk to each other and work together without any confusion.

In this report, we will learn what ROS is, what a ROS workspace means, and how topics, messages, and services work in ROS. We will also understand the main parts of ROS like nodes, roscore, roslaunch, rosmaster, and rosrund. This will give us a clear idea of how ROS helps in creating and running robots efficiently.

What is ROS?

ROS stands for Robot Operating System. It is not an actual operating system like Windows or Linux. Think of it more like a big toolbox and a messaging system that helps different parts of a robot talk to each other and work together. ROS gives you common building blocks so you do not waste time redoing the same basic stuff for every robot project. It helps you focus on the cool part, like making the robot smart and useful.

ROS is a set of tools, libraries, and conventions that let you build robot software as smaller pieces that communicate. These pieces can run on different computers and use different programming languages, but they can still talk to each other easily. ROS is widely used because it is open source, flexible, and supported by a big community.

Why people use ROS

- It stops you from reinventing the wheel. Reuse existing code and packages.
- It makes programs modular so you can swap parts without breaking everything.
- It works across machines. Your sensor can be on one computer and your planner on another.
- It supports multiple languages like Python and C plus plus, so you can use the right tool for the job.
- It has a strong community and lots of ready made packages for common sensors and robots.

Main components of ROS

Nodes

Nodes are small programs that do one job.

Example: a camera node reads images. A motor node drives wheels. A planner node decides where to go. Nodes keep things tidy and focused.

Topics and messages

Topics are channels or group chats where nodes send information. Messages are the actual content sent on those channels.

Example: a camera node publishes image messages to the topic called /camera/image. Any other node that needs images subscribes to that topic. This is great for streaming sensor data because many listeners can get the same data at once.

Services

Services let one node ask another node for something and wait for a reply. This is like calling a function on another computer. Use services when you need a request and a direct response, for example to ask a robot arm to move to a specific position and wait for confirmation.

Parameter server

A simple key value store where nodes can read or save configuration values. Think of it as global settings the robot can use at startup or while running.

Bags

A bag is a file that records ROS messages over time. You can record sensor data once and replay it later. This is awesome for debugging and testing without needing the real robot every time.

The master and core tools

ROS needs something that helps nodes find each other and register what they publish or provide. That role is filled by the master. In ROS version 1 you often run roscore which starts that central service. In ROS version 2 the architecture is different but the idea is the same: a system that helps nodes discover each other.

Common commands you will hear about:

roscore- starts the basic ROS services in ROS 1.

roslaunch- runs a single node from a package.

roslaunch- runs multiple nodes and sets things up using a single configuration file. It is great for starting complex systems.

roslaunch- records or plays back topics.

roslaunch- is a 3D visualization tool. It helps you see sensor data, robot pose, and more in a visual way.

roslaunch- is a simulator where you can run robots in virtual worlds to test without breaking hardware.

Workspace and packages

A workspace is where you build and keep your ROS code. It organizes packages and dependencies. A package is the basic unit of code distribution. A package can contain nodes, message types, launch files, configuration, and documentation. Packages make sharing and reusing code easy.

Language and platform support

ROS supports multiple languages, most commonly Python and C plus plus. It runs best on Linux, especially Ubuntu, because most community tutorials and tools assume that setup. You can work with ROS on other platforms but the smoothest path is usually Linux.

Community and ecosystem

ROS has a big community. People share packages for sensors, mapping, navigation, computer vision, simulation, and more. That means for many common parts you do not need to code from scratch. The community also provides tutorials, Q and A sites, and example projects.

Strengths and trade offs

Strengths

- Speeds up development.
- Encourages modular design.
- Huge library of existing packages.
- Works across multiple machines and languages.

Trade offs

- There is a learning curve at first, especially if you are new to Linux.
- ROS 1 and ROS 2 have different architectures, so you should pick the right one for your needs.
- For very time critical low level control, you still need careful design because ROS adds abstraction layers

What is ROS 2

ROS 2 is the next generation of the Robot Operating System. It was created to fix some of the limitations of the original ROS and to make it more powerful, secure, and suitable for industrial and real-world applications.

Here's what makes ROS 2 special:

- It is built on DDS (Data Distribution Service), which makes communication faster, safer, and more reliable.
- It supports real-time systems, which are crucial for robots that need to react instantly (like self-driving cars or drones).
- It works much better on multiple platforms, including Linux, Windows, and macOS.
- It provides better security features for protecting data and connections.
- It has improved multi-robot support, allowing several robots to coordinate easily.

Basically, ROS 2 takes everything that made ROS great and upgrades it for the modern robotics world. It's the future of robotic software development and is quickly becoming the standard for new projects.

References:

- <https://www.clearpathrobotics.com/assets/guides/melodic/ros/Intro%20to%20the%20Robot%20Operating%20System.html>
- <https://www.geeksforgeeks.org/operating-systems/introduction-to-ros-robot-operating-system/>

What is a ROS Workspace?

In ROS (Robot Operating System), all your projects, packages, and codes are organized inside something called a **workspace**.

A ROS workspace is basically a special folder on your computer where you create, build, and manage your robot programs. It helps keep everything neat, structured, and easy to work with, like a personal workshop for your robot code.

A ROS workspace is a directory that contains one or more ROS packages. A workspace is like a project folder where you can organize your ROS packages and build them together.

When you create a new workspace, ROS automatically sets up a few directories inside it:

```
catkin_ws/  
├── build/  
├── devel/  
└── src/
```

src (source) - this is where you put your ROS packages. (source code)

build - this is where ROS builds your packages. ROS uses this folder when compiling your code. (compilation process)

devel (development) - this is where ROS installs your packages, where ROS keeps the built files and executables so you can run them. (ready-to-use files)

ROS Packages

A ROS package is like a small project or folder that holds everything needed for a specific robot function. It includes code, data, and setup files. Think of packages as the building blocks of your robot, each one handles a different task.

- A package can have one or more nodes, which are small programs that do specific jobs.
- It can also include launch files, message definitions, and configuration files.
- Every package must have a unique name written in lowercase and separated by underscores (like `my_robot_package`).
- Each package also needs a `package.xml` file- this file gives information about the package, like its name, author, and dependencies (what other packages it needs to work).

A package contains:

- Source code (your actual programs or “nodes”)
- Configuration files
- Message definitions
- Launch files
- Data files

Every package must have two important files:

1. `package.xml` - describes your package (name, author, dependencies).
2. `CMakeLists.txt` - tells ROS how to build your code.

What is Catkin?

ROS uses something called **Catkin** as its official build system.

Catkin helps compile packages, handle dependencies, and make sure everything in your workspace works together smoothly.

Fun fact: the name “catkin” comes from the little flower clusters found on willow trees, a nod to Willow Garage, the company that created ROS.

Creating a Workspace

To make your first workspace in Linux, you can run these commands in your terminal:

```
mkdir -p ~/catkin_ws/src
```

```
cd ~/catkin_ws
```

```
catkin build
```

Explanation:

- `mkdir -p ~/catkin_ws/src` creates the workspace and the source folder inside it.
- `cd ~/catkin_ws` moves you into that folder.
- `catkin build` builds your workspace and sets everything up.

Once the build finishes, you’ll see new folders: `build`, `devel`, and maybe `logs`. Your workspace is now ready for use.

Creating a ROS Package

To make your own package, use this command inside the `src` folder of your workspace:

```
cd ~/catkin_ws/src
```

```
catkin_create_pkg my_robot_pkg rospy std_msgs
```

Here:

- `my_robot_pkg` is the name of your package.
- `rospy` and `std_msgs` are dependencies (libraries that your package needs).

After running it, ROS will automatically create:

- A folder for your package
- A `CMakeLists.txt` file
- A `package.xml` file

You can then start adding your nodes, launch files, and other resources.

Building a ROS Package

Once your package is ready, go back to your workspace root and build it:

```
cd ~/catkin_ws
```

```
catkin build
```

This command compiles all the packages inside your workspace and places the ready-to-use files in the devel directory.

Difference between ROS and ROS2 workspace

Feature	ROS1	ROS2
Build System	catkin	colcon
Workspace Name	catkin_ws	ros2_ws
Key folders	src,build,devel	src,build,install,log
Build command	catkin build	colcon build

References:

- <https://ros-learnings.hashnode.dev/comprehension-of-ros-workspace-packages>
- <https://robodev.blog/ros-packages-and-workspace>

What are ROS topics, messages and services?

ROS Topics

A topic in ROS is like a public message board that different parts of a robot use to talk to each other. Think of it like this, one node can publish information on a topic (like posting something on the board), and another node can subscribe to that topic to read it. This is how data moves around inside a robot.

Each topic has its own name, and it's mainly used to send things like:

- Sensor data (for example, readings from a camera or LiDAR)
- Control commands (like movement instructions)
- Status updates (like battery level or position)

You can see the list of active topics in your system by running:
`rostopic list`

And to get details about a specific topic:

`rostopic info <topic_name>`

(Remember to start roscore first before running these!)

When to Use Topics

Topics are perfect when you need one-way, real-time communication, data just keeps flowing in one direction.

For example:

- Sending sensor readings constantly
- Publishing robot speed or position updates
- Controlling motors or arms in real time

You can also have many publishers and many subscribers on one topic, meaning multiple nodes can send or receive data at the same time.

In short, topics are how ROS keeps your robot's parts talking smoothly, like a constant stream of updates keeping everything in sync.

ROS Services

In ROS, a service is a way for one node to ask another node for something and get an immediate response. Think of it like a question-and-answer system:

- One node acts as a Service Server, waiting for requests and sending responses.
- Another node acts as a Service Client, sending requests and receiving responses.

For example, a robot might have a service that provides its current battery level, or one that resets its position in a simulation. The client sends a request, and the server gives back an answer, fast and direct.

When to Use Services

Services are ideal for situations where you need specific, one-time interactions, rather than a continuous flow of data. They are perfect for:

- Requesting a single piece of information (like the robot's current status).
- Triggering a specific action that doesn't happen continuously.

How Services Work

Each service has two main parts:

1. **Request** – the information or action you are asking for.

2. **Response** – the answer or result returned by the server.

This makes services different from topics, which are more like a live broadcast, constantly sending data to anyone listening. Services are more controlled and on-demand, just like asking a friend a question and waiting for their reply.

Listing and Inspecting Services

To see all services available in your ROS2 system:

```
rosservice list
```

To get detailed info about a specific service:

```
rosservice info <service_name>
```

Features	Topics	Servies
Type of Communication	Continuous data stream	One-time request & response
Example	Robot constantly sending its speed or sensor data	Asking the robot for its battery level
Flow Direction	Publisher to Subscriber (one-way)	Client ↔ Server (two-way)
Timing	Happens continuously in real time	Happens only when requested
Use Case	Live updates (like video feed, sensor data, odometry)	On-demand info or action (like resetting a simulation, getting a map)
Analogy	Like a live news broadcast – data keeps coming	Like a phone call – you ask something, get a reply, and hang up
Speed	Fast & continuous	Slower but more structured
Message Direction	Only data going out	Data going both ways (request + response)

ROS Messages

In ROS, a message is the actual content that gets sent between nodes.

Think of a message like an article on a bulletin board, it has a specific structure, just like an article has a title, author, and content. In ROS, a message can contain different types of data, such as:

- Numbers (integers or floating points)
- Text (strings)
- Boolean values, arrays, and more

Messages and Topics

Topics are the channels through which nodes communicate.

- Every node publishing or subscribing to a topic must use the same message type.
- A topic is defined by:
 1. A name – the interface to access it
 2. A message definition – the structure of the data being sent

For example, if a robot's camera publishes images on a topic, all nodes subscribing to that topic must expect image messages in the defined format.

Messages and Services

Services are another way for nodes to communicate, but unlike topics, services are two-way: a request and a response.

- A service is defined by:
 1. A name – the interface to access it
 2. A service definition – which includes a request message and a response message

Think of it like sending a letter: the letter you send is the request, and the reply you get back is the response. Together, these two messages make up the service definition.

Listing ROS Messages

You can see the available message types in your system with:

```
rosmmsg list
```

To get more details about a specific message type:

```
rosmmsg show <message_type>
```

How ROS Messages Work in Code

Technically, ROS messages are defined first, and then the build system generates code so you can use them directly in your programs (C++, Python, etc.).

- When you build your workspace with `catkin_make` (or `colcon` in ROS2), ROS parses the message definitions and creates source files automatically.
- These generated files include headers or modules that your nodes can import to send or receive messages.

This system ensures that all nodes understand the exact structure of the data being shared, which keeps communication smooth and error-free.

Why ROS Messages Are Important

Messages are the core of communication in ROS.

- They define what information nodes need to share.
- Without messages, topics and services would have no content to transmit.
- They make sure that all nodes “speak the same language” so your robot’s components can work together seamlessly.

References:

- <https://www.theconstruct.ai/ros2-how-to-ros2-topics-vs-services-vs-actions-5/>
- <https://medium.com/@sung.minner/ros-concepts-understanding-topics-nodes-messages-and-services-b53491dfa12e>
- <https://roboticsbackend.com/what-is-a-ros-message/>

What are ROS nodes?

What is a ROS Node?

A ROS Node is one of the most important parts of the Robot Operating System. It is an independent program or process that performs a specific function in a robotic system. Instead of writing one long program to control everything, ROS allows us to divide the robot's tasks into smaller, manageable units called nodes. These nodes communicate and work together to make the robot operate efficiently.

Understanding the Concept

Think of a ROS node as a building block of your robot's brain. Each node is responsible for one part of the robot's job. For example:

- One node can read data from a camera or sensor.
- Another node can plan the robot's path or movement.
- Another can control the motors and move the robot.

By dividing the robot's functions into nodes, developers can manage, update, and debug each part separately without affecting the others. This structure also allows multiple nodes to run at the same time, each handling a specific task.

How Nodes Work Together

In a ROS-based system, multiple nodes run simultaneously and communicate with each other.

For example:

- A **camera node** captures images from the environment.
- An **image processing node** analyses these images and detects obstacles.
- A **motion planning node** decides how the robot should move.
- A **motor control node** sends commands to the wheels.

Each node performs its own function but can exchange data with others using ROS communication tools such as topics, services, and actions.

Advantages of Using Nodes

1. **Simplifies Complex Systems**
Splitting a large application into smaller nodes makes it easier to design, understand, and maintain.
2. **Improves Fault Tolerance**
If one node fails or crashes, other nodes continue to run. This prevents the entire system from stopping.
3. **Supports Parallel Execution**
Multiple nodes can run at the same time, allowing the robot to perform several tasks simultaneously.
4. **Promotes Reusability**
A node written for one robot can be reused in another robot with minimal changes.
5. **Language Flexibility**
ROS nodes can be written in different programming languages, mainly Python and C++. This gives developers flexibility to choose the best language for each task.

Naming and Organization

Each node in ROS must have a unique name. This helps ROS identify and manage them properly.

Nodes are usually grouped inside packages, which contain related files like launch files, configurations, and scripts. You can run, stop, or monitor nodes individually, making it easier to test or modify a specific part of the robot's system.

Example

Imagine a mobile robot that needs to navigate a room:

- The **sensor node** collects distance data from sensors.
- The **mapping node** uses this data to create a map.
- The **path planning node** figures out the best route.
- The **control node** sends movement commands to the motors.

These nodes continuously communicate with each other, allowing the robot to move intelligently without crashing into obstacles.

Understanding Core ROS Commands: roscore, roslaunch, rosmaster, and rosrun

When working with the Robot Operating System (ROS), there are a few core commands and tools that form the backbone of how nodes communicate and function together. These tools make it possible to start, organize, and manage your robot's software system efficiently.

Let's break down what each one does and why it matters.

1. roscore

The roscore command is the starting point of any ROS system.

It brings up the main background services that allow all nodes to communicate and share data.

When you run roscore, it automatically starts three key components:

- **ROS Master** – helps different nodes find and connect with each other.
- **Parameter Server** – stores and shares configuration values or parameters used by nodes.
- **rosout Logging Node** – collects and displays logs or messages from nodes (like a debugging console).

You can think of roscore as the “central nervous system” of ROS. Without it, nodes cannot talk to one another. It must be running before you start any other node or process.

If you use roslaunch, it automatically starts roscore if it's not already running.

2. roslaunch

The roslaunch command makes it easy to start multiple ROS nodes together, instead of running each one separately.

It uses .launch files, which are XML files that describe:

- Which nodes to start.
- What parameters to set.
- Any dependencies or configurations needed.

You can think of a launch file like a “director's script” for your robot, it tells ROS what to start, where, and how.

For example, a single .launch file can start your camera node, motion controller, and visualization tool all at once.

roslaunch also allows you to:

- Run nodes on multiple computers using SSH.
- Automatically restart a node if it crashes.
- Manage large projects more easily by grouping related launch files.

In short, roslaunch saves time, keeps systems organized, and simplifies the process of running complex robotic setups.

3. rosmaster

The ROS Master acts as the central communication hub for all nodes.

It doesn't send or receive messages itself, instead, it helps nodes find each other and establish direct communication links.

Here's how it works:

- When a node starts, it registers itself with the master.
- If a node wants to publish or subscribe to a topic, it asks the master for information about who else is publishing or subscribing to that topic.
- Once the connection is made, the nodes exchange data directly without involving the master again.

So basically, the master is like a “matchmaker”, it introduces nodes to each other and then steps out of the way. If the master stops running, new nodes can’t find each other, though existing ones may continue communicating until they’re restarted.

4. rosrun

The rosrun command is the simplest way to start an individual ROS node from a package.

It allows you to run a single executable node without needing a launch file.

The syntax is:

```
roslaunch <package_name> <node_name>
```

For example, if you just want to quickly test one specific node, say, a sensor reader or motor controller you can use roslaunch instead of setting up a full .launch file.

It’s perfect for small tasks, debugging, or quick experiments, while roslaunch is better for running multiple nodes together.

References:

- <https://wiki.ros.org/ROS/Tutorials/UnderstandingNodes#Nodes>
- <https://roboticsbackend.com/what-is-a-ros-node/>

*******END*******